



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 22: Patrones Estructurales (parte 1)

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

23 de Octubre, 2025

Patrones estructurales

Ya vimos los patrones creacionales:

- Estos patrones nos permiten crear objetos de manera flexible y permiten reusar código existente.

Ya vimos los patrones creacionales:

- Estos patrones nos permiten crear objetos de manera flexible y permiten reusar código existente.

Ahora veremos los patrones estructurales:

- Estos patrones nos permiten combinar objetos y clases en estructuras más grandes, manteniendo estas estructuras flexibles y eficientes.

Actividad

Simulando la vida de un honguito

Se quiere simular la vida de honguitos. Todo honguito nace, se reproduce y muere siguiendo distintas condiciones que dependen del tipo de honguito.

Simulando la vida de un honguito

Se quiere simular la vida de honguitos. Todo honguito nace, se reproduce y muere siguiendo distintas condiciones que dependen del tipo de honguito.

Honguito clásico:

- Sobrevive solo si tiene 2 o 3 vecinos.
- Cada turno lanzan esporas a sus casillas vecinas.
- Si caen exactamente 3 esporas en una casilla vacía nace un honguito.

Simulando la vida de un honguito

Se quiere simular la vida de honguitos. Todo honguito nace, se reproduce y muere siguiendo distintas condiciones que dependen del tipo de honguito.

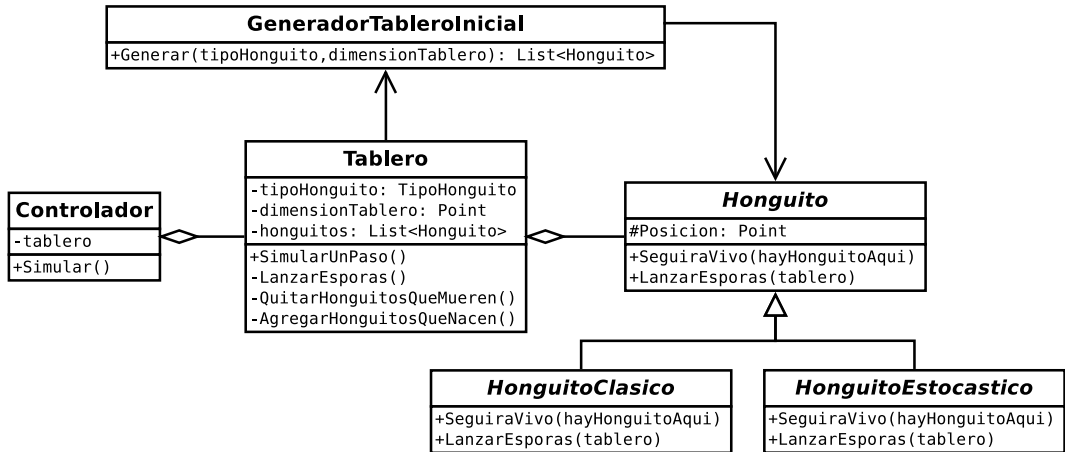
Honguito clásico:

- Sobrevive solo si tiene 2 o 3 vecinos.
- Cada turno lanzan esporas a sus casillas vecinas.
- Si caen exactamente 3 esporas en una casilla vacía nace un honguito.

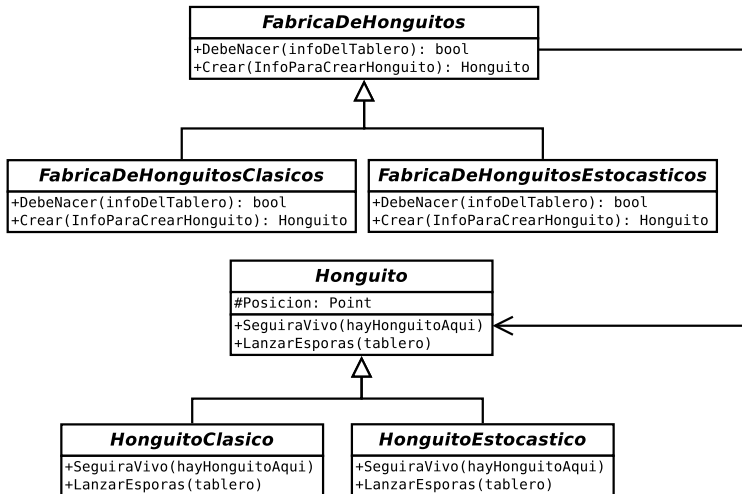
Honguito estocástico:

- En cada turno, sobrevive con probabilidad 0.2.
- Cada turno lanzan 15 esporas que se reparten siguiendo una distribución normal.
- La probabilidad de que nazca un honguito depende de la cantidad de esporas que caen en una casilla vacía.

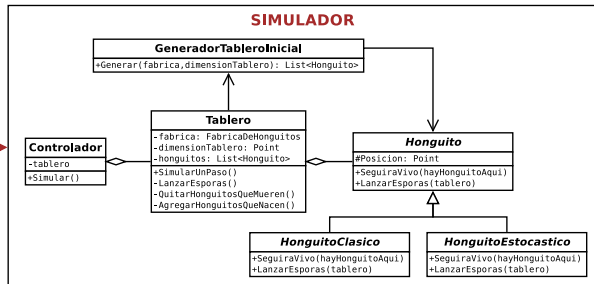
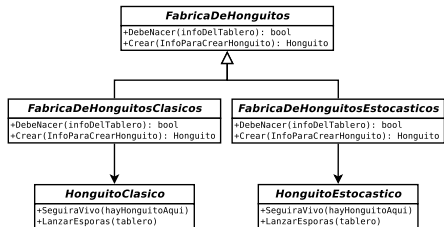
Simulando la vida de un honguito



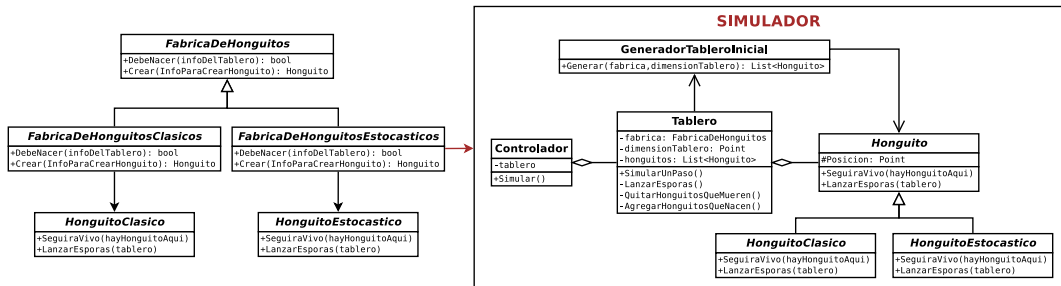
Simulando la vida de un honguito



Simulando la vida de un honguito



Simulando la vida de un honguito



¿Qué pasa si queremos agregar un nuevo tipo de honguito?

Simulando la vida de un honguito

Problema: Comienzan a aparecer honguitos que son mezclas entre honguitos clásicos y estocásticos. Por ejemplo, aparece un honguito que nace y lanza esporas como un honguito clásico pero que muere siguiendo la regla de un honguito estocástico.

Simulando la vida de un honguito

Problema: Comienzan a aparecer honguitos que son mezclas entre honguitos clásicos y estocásticos. Por ejemplo, aparece un honguito que nace y lanza esporas como un honguito clásico pero que muere siguiendo la regla de un honguito estocástico.

Nuevo honguito:

- ~~Sobrevive solo si tiene 2 o 3 vecinos.~~ En cada turno, sobrevive con prob 0.2.
- Cada turno lanzan esporas a sus casillas vecinas.
- Si caen exactamente 3 esporas en una casilla vacía nace un honguito.

Simulando la vida de un honguito

Problema: Comienzan a aparecer honguitos que son mezclas entre honguitos clásicos y estocásticos. Por ejemplo, aparece un honguito que nace y lanza esporas como un honguito clásico pero que muere siguiendo la regla de un honguito estocástico.

Nuevo honguito:

- ~~Sobrevive solo si tiene 2 o 3 vecinos.~~ En cada turno, sobrevive con prob 0.2.
- Cada turno lanzan esporas a sus casillas vecinas.
- Si caen exactamente 3 esporas en una casilla vacía nace un honguito.

Programamos este nuevo honguito, pero notamos que más y más mezclas aparecen.

Simulando la vida de un honguito

Problema: Comienzan a aparecer honguitos que son mezclas entre honguitos clásicos y estocásticos. Por ejemplo, aparece un honguito que nace y lanza esporas como un honguito clásico pero que muere siguiendo la regla de un honguito estocástico.

Nuevo honguito:

- ~~Sobrevive solo si tiene 2 o 3 vecinos.~~ En cada turno, sobrevive con prob 0.2.
- Cada turno lanzan esporas a sus casillas vecinas.
- Si caen exactamente 3 esporas en una casilla vacía nace un honguito.

Programamos este nuevo honguito, pero notamos que más y más mezclas aparecen. Rápidamente llegamos a la siguiente conclusión:

- Si hay N formas de nacer, M formas de lanzar esporas y O formas de morir.
- Entonces hay $N \times M \times O$ tipos posibles de honguitos.

Simulando la vida de un honguito

Problema: Comienzan a aparecer honguitos que son mezclas entre honguitos clásicos y estocásticos. Por ejemplo, aparece un honguito que nace y lanza esporas como un honguito clásico pero que muere siguiendo la regla de un honguito estocástico.

Nuevo honguito:

- ~~Sobrevive solo si tiene 2 o 3 vecinos.~~ En cada turno, sobrevive con prob 0.2.
- Cada turno lanzan esporas a sus casillas vecinas.
- Si caen exactamente 3 esporas en una casilla vacía nace un honguito.

Programamos este nuevo honguito, pero notamos que más y más mezclas aparecen. Rápidamente llegamos a la siguiente conclusión:

- Si hay N formas de nacer, M formas de lanzar esporas y O formas de morir.
- Entonces hay $N \times M \times O$ tipos posibles de honguitos.

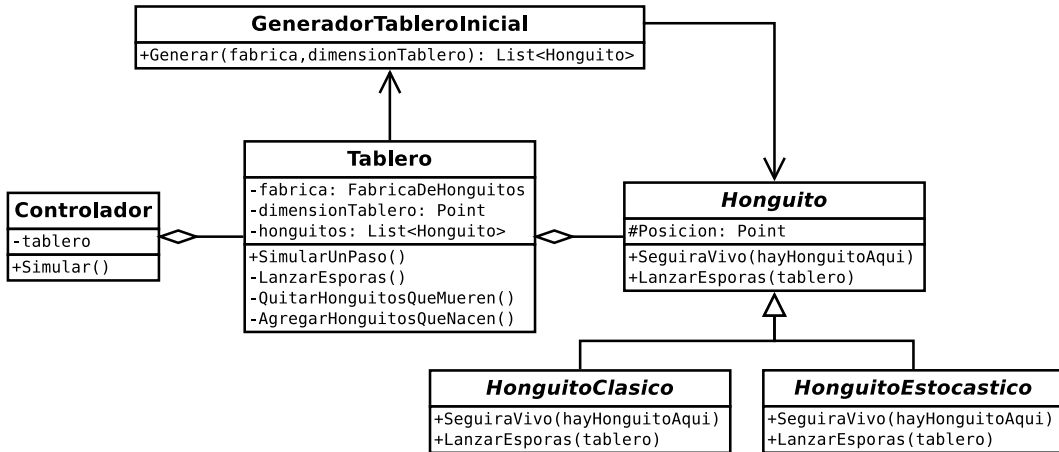
¡No podemos tener una clase por honguito!

Simulando la vida de un honguito

¿Cómo logramos que el simulador funcione para los $N \times M \times O$ honguitos posibles?

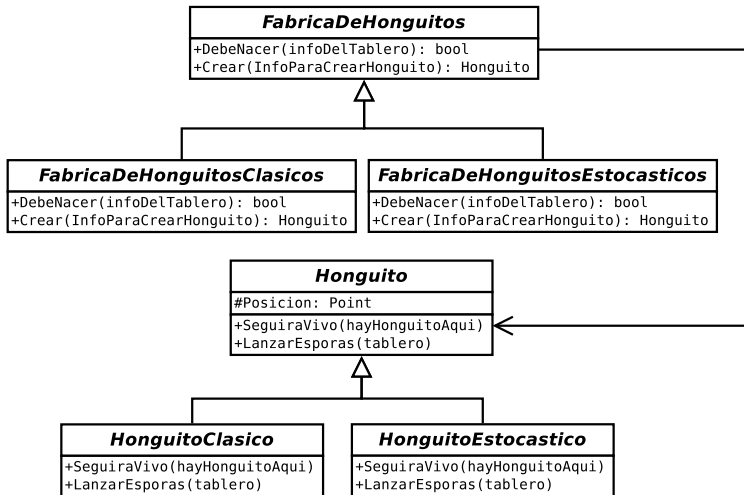
Simulando la vida de un honguito

¿Cómo logramos que el simulador funcione para los $N \times M \times O$ honguitos posibles?

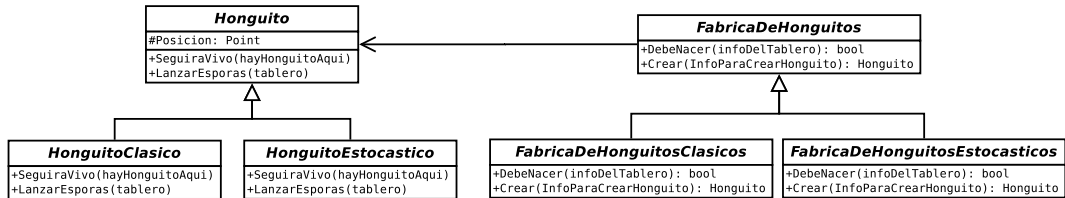


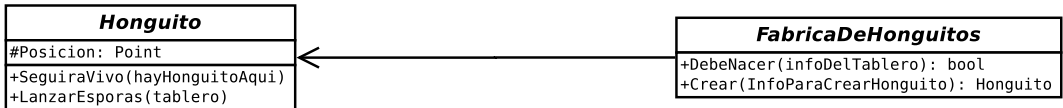
Simulando la vida de un honguito

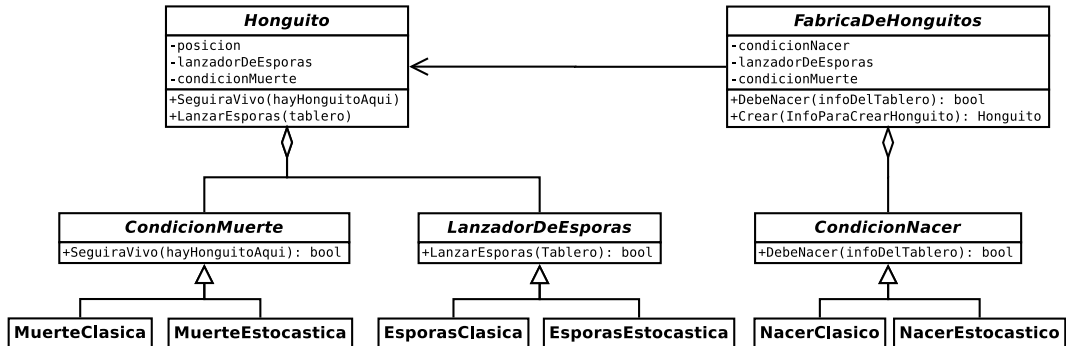
¿Cómo logramos que el simulador funcione para los $N \times M \times O$ honguitos posibles?

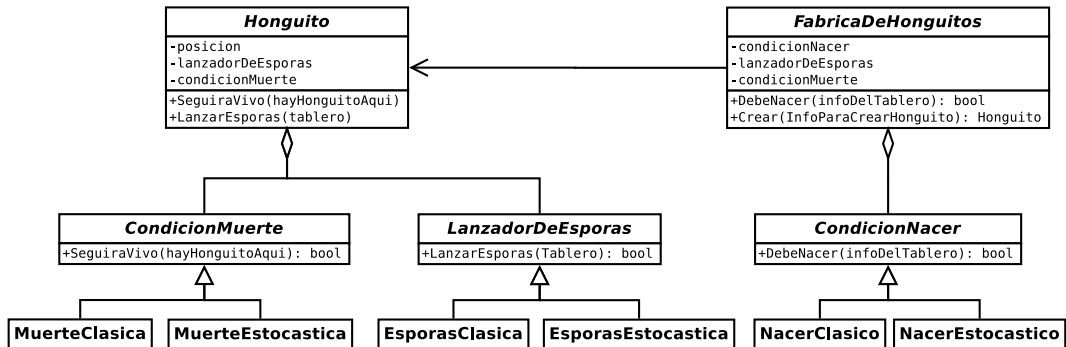


Solución









Con este enfoque solo debemos implementar $N + M + O$ clases.

Luego en el main construimos un honguito a la medida:

```
6 CondicionNacer nace = new CondicionNacerClasica();
7 LanzadorDeEsporas esporas = new LanzadorDeEsporasClasico();
8 CondicionMuerte muere = new CondicionMuerteClasica();
9
10 FabricaDeHonguito fabrica = new FabricaDeHonguito(nace, esporas, muere);
11
12 Controlador controlador = new Controlador(fabrica, dimTab);
13 controlador.Simular();
```

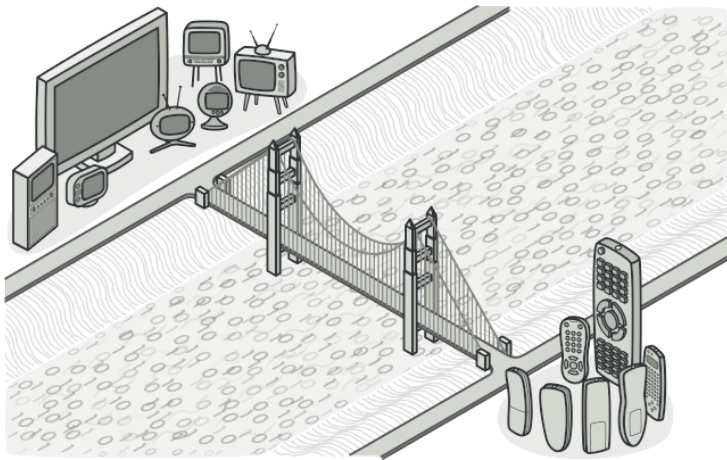
```
2 public class Honguito {
3     private Point Posicion;
4     private LanzadorDeEsporas esporas;
5     private CondicionMuerte muerte;
6     public Honguito(Point posicion,
7         LanzadorDeEsporas esporas, CondicionMuerte muerte) {
8         Posicion = posicion;
9         this.esporas = esporas;
10        this.muerte = muerte;
11    }
12    public bool SeguirVivoEnElSiguienteTurno(bool[,] hayHonguitos)
13        => muerte.SeguirVivoEnElSiguienteTurno(Posicion, hayHonguitos);
14    public void LanzarEsporas(int[,] tablero)
15        => esporas.LanzarEsporas(Posicion, tablero);
16    public void MarcarHonguitoEnTablero(bool[,] hayHonguitos)
17        => hayHonguitos[Posicion.X, Posicion.Y] = true;
18 }
```

Solución

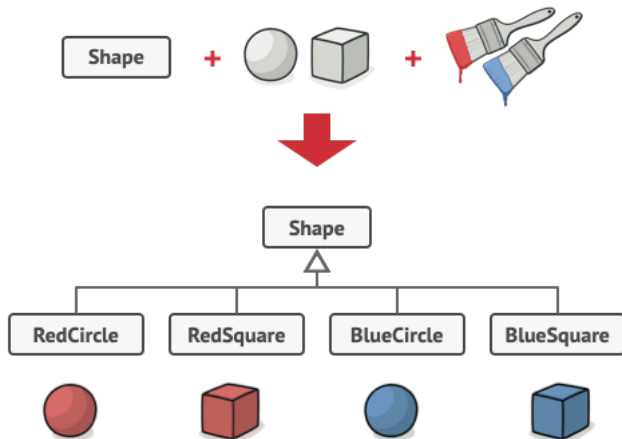
```
2 public class FabricaDeHonguitos{
3     private CondicionNacer nacer;
4     private LanzadorDeEsporas esporas;
5     private CondicionMuerte muerte;
6
7     public FabricaDeHonguitos(CondicionNacer nacer,
8         LanzadorDeEsporas esporas, CondicionMuerte muerte){
9         this.nacer = nacer;
10        this.esporas = esporas;
11        this.muerte = muerte;
12    }
13
14    public bool DebeNacer(Point posicion, int[,] conteoEsporas)
15        => nacer.DebeNacer(posicion, conteoEsporas);
16    public Honguito Crear(Point posicion, Point dimensionTablero)
17        => new Honguito(posicion, esporas, muerte);
18 }
```

Bridge

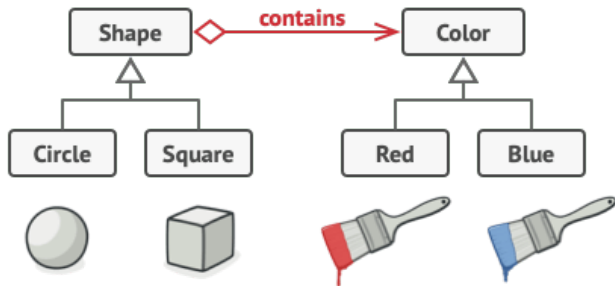
Bridge



Bridge



Bridge



Bridge

